



CSS Variables (var() Function)

Master CSS custom properties for dynamic, maintainable stylesheets

What are CSS Variables?

CSS Variables (officially called CSS Custom Properties) are reusable values that can be defined once and used throughout your stylesheets. They allow you to store colors, sizes, spacing, and any other CSS value in a variable, making your code more maintainable and flexible.

CSS Variables are fundamentally different from preprocessor variables (like SASS) because they are dynamic and can be changed at runtime using JavaScript or by using different selectors.

Browser Support: All modern browsers (Chrome 49+, Firefox 31+, Safari 9.1+, Edge 15+) support CSS Variables. IE 11 does not support them.

Core Concepts

Defining CSS Variables

CSS Variables are defined using the `--variable-name` syntax inside a selector:

```
Syntax: --variable-name: value;  
  
Example:  
.root {  
  --primary-color: #667eea;  
  --spacing: 16px;  
  --border-radius: 8px;  
}
```

Using CSS Variables with var()

Use the `var()` function to reference a CSS Variable:

```
Syntax: property: var(--variable-name);  
  
With Fallback: property: var(--variable-name, fallback-value);  
  
Example:  
.button {  
  background: var(--primary-color);  
  padding: var(--spacing);  
  border-radius: var(--border-radius);  
}
```

Variable Scope

CSS Variables defined in `:root` are global and accessible everywhere. Variables defined in other selectors are only accessible within that selector's scope and its descendants.

```
/* Global scope */  
:root {  
  --primary-color: blue;  
}  
  
/* Local scope */  
.button {  
  --button-size: 16px;  
}  
  
/* Can override global variables */  
.dark-theme {  
  --primary-color: lightblue;  
}
```

Inheritance

CSS Variables are inherited from parent to child elements, just like other CSS properties:

```
:root { --color: red; }  
  
.parent {  
  color: var(--color); /* red */  
}  
  
.parent .child {  
  color: var(--color); /* Also red - inherited */  
}
```

Defining and Using CSS Variables

Naming Conventions

Follow these best practices for naming CSS Variables:

- Start with `--` (two hyphens, required)
- Use lowercase with hyphens: `--primary-color` not `--primaryColor`
- Be descriptive: `--button-hover-color` is better than `--c1`
- Group related variables: `--spacing-sm`, `--spacing-md`, `--spacing-lg`
- Consider namespace: `--color-primary`, `--font-size-base`

Common Variable Categories

```
/* Colors */  
--primary-color: #667eea;  
--secondary-color: #764ba2;  
--accent-color: #ff6b6b;  
  
/* Spacing */  
--spacing-xs: 4px;  
--spacing-sm: 8px;  
--spacing-md: 16px;  
--spacing-lg: 24px;  
--spacing-xl: 32px;  
  
/* Typography */  
--font-size-sm: 12px;  
--font-size-base: 16px;  
--font-size-lg: 20px;  
--font-family-base: 'Segoe UI', sans-serif;  
--line-height: 1.6;  
  
/* Sizing */  
--border-radius: 8px;  
--transition-speed: 0.3s;  
--shadow: 0 2px 8px rgba(0,0,0,0.1);
```

The var() Function

```
Syntax: var(--name) | var(--name, fallback)  
  
Example with fallback:  
color: var(--text-color, black);  
  
Fallback can be another var():  
padding: var(--custom-padding, var(--default-padding, 16px));  
  
Use in calculations:  
width: calc(100% - var(--sidebar-width));
```

🎨 Interactive CSS Variables Demos

Theme Switching with CSS Variables

Light ThemeDark Theme

Switch themes to see how CSS Variables dynamically update all colors throughout the page.

Color Palette Using Variables

Primary Color
`--primary-color`

Secondary Color
`--secondary-color`

Text Color
`--text-color`

Background
`--background-color`

Interactive Variable Editor

Primary Color:

Border Radius (px):

Spacing (px):

Preview: These values update in real-time using CSS Variables!

Box with dynamic variables

Practical Use Cases & Patterns

1. Consistent Color Palette

```
:root {  
  --color-primary: #667eea;  
  --color-secondary: #764ba2;  
  --color-success: #48bb78;  
  --color-error: #f56565;  
  --color-warning: #ed8936;  
}  
  
.btn-primary { background: var(--color-primary); }  
.btn-success { background: var(--color-success); }  
.btn-error { background: var(--color-error); }
```

2. Responsive Spacing System

```
:root {  
  --spacing-unit: 8px;  
  --spacing-xs: calc(var(--spacing-unit) * 0.5);  
  --spacing-sm: var(--spacing-unit);  
  --spacing-md: calc(var(--spacing-unit) * 2);  
  --spacing-lg: calc(var(--spacing-unit) * 3);  
}  
  
@media (max-width: 768px) {  
  :root {  
    --spacing-unit: 6px;  
  }  
  /* All spacing automatically scales down */  
}
```

3. Typography System

```
:root {  
  --font-family-base: 'Segoe UI', sans-serif;  
  --font-size-base: 16px;  
  --line-height-base: 1.5;  
}  
  
body {  
  font-family: var(--font-family-base);  
  font-size: var(--font-size-base);  
  line-height: var(--line-height-base);  
}  
  
h1 { font-size: calc(var(--font-size-base) * 2.5); }  
h2 { font-size: calc(var(--font-size-base) * 2); }  
h3 { font-size: calc(var(--font-size-base) * 1.5); }
```

4. Theme Switching (Light/Dark Mode)

```
/* Light theme */  
:root {  
  --bg: white;  
  --text: #333;  
  --border: #ddd;  
}  
  
/* Dark theme */  
:root.dark {  
  --bg: #1a1a1a;  
  --text: #fff;  
  --border: #333;  
}  
  
/* Variables used everywhere */  
body {  
  background: var(--bg);  
  color: var(--text);  
}
```

5. Component-Specific Variables

```
.card {  
  --card-padding: var(--spacing-lg);  
  --card-border-radius: var(--border-radius);  
  --card-shadow: 0 2px 8px rgba(0,0,0,0.1);  
  padding: var(--card-padding);  
  border-radius: var(--card-border-radius);  
  box-shadow: var(--card-shadow);  
}  
  
.button {  
  --btn-padding: var(--spacing-sm) var(--spacing-md);  
  --btn-color: var(--primary-color);  
  padding: var(--btn-padding);  
  background: var(--btn-color);  
}
```

6. Dynamic Calculations

```
:root {  
  --base-size: 10px;  
  --gutter: 2rem;  
}  
  
.grid {  
  /* Column width dynamically calculated */  
  width: calc(100% - var(--gutter));  
  margin: var(--gutter);  
}  
  
.item {  
  /* Responsive sizing */  
  padding: calc(var(--base-size) * 2);  
}
```

JavaScript Integration

Reading CSS Variables

```
/* Get a CSS Variable value */  
const primaryColor = getComputedStyle(document.documentElement)  
  .getPropertyValue('--primary-color');  
  
console.log(primaryColor); // '#667eea'
```

Setting CSS Variables Dynamically

```
/* Change a global CSS Variable */  
document.documentElement.style  
  .setProperty('--primary-color', '#ff6b6b');  
  
/* Change for a specific element */  
document.querySelector('.button').style  
  .setProperty('--btn-color', 'blue');
```

Complete Theme Switcher Example

```
function setTheme(themeName) {  
  if (themeName === 'dark') {  
    document.documentElement.classList.add('dark-theme');  
    localStorage.setItem('theme', 'dark');  
  } else {  
    document.documentElement.classList.remove('dark-theme');  
    localStorage.setItem('theme', 'light');  
  }  
}  
  
// Restore saved theme  
const savedTheme = localStorage.getItem('theme') || 'light';  
setTheme(savedTheme);
```

Advanced Techniques

Nested/Scoped Variables

```
.component {  
  --primary: blue;  
  color: var(--primary);  
}  
  
.component.variant {  
  --primary: red;  
  /* Same CSS, different color */  
  color: var(--primary);  
}
```

Fallback Chains

```
/* Multiple fallbacks */  
color: var(--custom-color, var(--theme-color, var(--default-color, blue)));  
  
/* If --custom-color doesn't exist, try --theme-color */  
/* If --theme-color doesn't exist, try --default-color */  
/* If all fail, use blue */
```

Using Variables in Media Queries

```
:root {  
  --column-count: 3;  
}  
  
@media (max-width: 1024px) {  
  :root { --column-count: 2; }  
}  
  
@media (max-width: 768px) {  
  :root { --column-count: 1; }  
}  
  
.grid {  
  grid-template-columns: repeat(var(--column-count), 1fr);  
  /* Automatically adapts to screen size */  
}
```

Conditional Variables

```
:root {  
  --is-dark-mode: 0; /* false */  
}  
  
:root.dark {  
  --is-dark-mode: 1; /* true */  
}  
  
/* Note: Variables don't support if/else */  
/* Use calc() creatively or class-based switching */
```

Best Practices

- Define variables in :root for global scope:** Makes them accessible everywhere in your stylesheet.
- Use meaningful names:** `--spacing-md` is clearer than `--s2`.
- Use group related variables:** Keep color, spacing, and font variables organized together.
- Provide fallbacks:** Use `var(--name, fallback)` for better browser compatibility.
- Use comment your variables:** Add comments explaining what each variable is for.
- Use CSS variables with preprocessors:** Combine SASS/LESS with CSS Variables for maximum power.
- Keep variables simple:** Don't create overly complex variable hierarchies.
- Test theme switching:** Ensure all elements update correctly when variables change.

CSS Variables vs Preprocessor Variables

Feature	CSS Variables	SASS Variables
Dynamic	Yes - can change at runtime	No - compiled at build time
Scoped	Yes - CSS cascade applies	No - global or lexical scope
Inherited	Yes - like CSS properties	No - requires manual passing
Changed with JS	Yes - at runtime	No - requires recompile
Browser Support	Modern browsers only	All browsers (compiled to CSS)
Browser DevTools	Visible and inspectable	Not visible (compiled)

Browser Compatibility

Modern Browsers: Chrome 49+, Firefox 31+, Safari 9.1+, Edge 15+ all support CSS Variables fully.

IE 11: Does NOT support CSS Variables. You must provide fallbacks or alternative styling.

Mobile: iOS Safari 9.3+, Android Chrome support CSS Variables.

Providing Fallbacks for Older Browsers

```
.button {  
  /* Fallback for unsupported browsers */  
  background-color: #667eea;  
  /* CSS Variables for modern browsers */  
  background-color: var(--primary-color);  
}
```

Quick Reference Guide

Define Variables

```
:root {  
  --variable-name: value;  
}
```

Use Variables

```
selector {  
  property: var(--variable-name);  
}
```

With Fallback

```
selector {  
  property: var(--variable-name, fallback-value);  
}
```

Common Variable Template

```
:root {  
  /* Colors */  
  --color-primary: #667eea;  
  --color-secondary: #764ba2;  
  
  /* Spacing */  
  --spacing-sm: 8px;  
  --spacing-md: 16px;  
  --spacing-lg: 24px;  
  
  /* Typography */  
  --font-size-base: 16px;  
  --line-height: 1.5;  
  
  /* Utilities */  
  --border-radius: 8px;  
  --transition: 0.3s ease;  
}
```